

NDNSF-DI

ACK-driven ONNX placement and NDN pull-based distributed inference

NDNSF Project

Target design and current verification boundary — August 2026

MODEL-FAMILY NEUTRAL

ACK-DRIVEN PLANNING

DATA-DRIVEN EXECUTION

The Problem: A Request Does Not Contain a Deployment

The application knows

- ▶ immutable model identity;
- ▶ task, input, options, objective;
- ▶ deadline and policy constraints.

Only the runtime can discover

- ▶ available Providers and GPU envelopes;
- ▶ queue, RTT, bandwidth, cache residency;
- ▶ a graph-valid split and role assignment.

Invariant

Provider discovery must precede the final split and placement decision.

Next: define the design goals and non-goals

Design Goals and Explicit Non-Goals

Goals

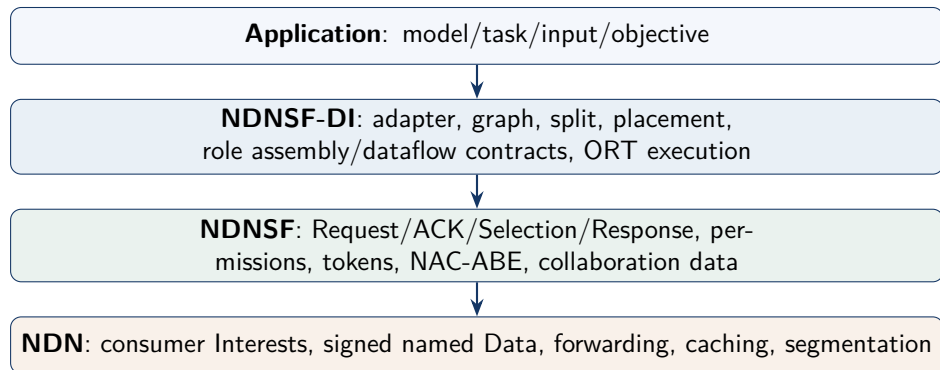
- ▶ one durable application invocation;
- ▶ replaceable placement policy;
- ▶ model-family-neutral adapters;
- ▶ graph- and capacity-safe splitting;
- ▶ authenticated, replay-safe execution;
- ▶ complete failure and recovery states.

Non-goals

- ▶ no model semantics in base NDNF;
- ▶ no global “all Providers ready” barrier;
- ▶ no trust in unsigned resource claims;
- ▶ no approximate cache-key matching;
- ▶ no performance claim from a control-plane smoke.

Next: place each responsibility in the correct layer

Ownership Boundary: NDNF Carries, NDNF-DI Interprets



Invariant

Cross-Provider activations and tensor collectives remain NDN Interest/Data; ONNX Runtime performs only Provider-local compute.

Core Vocabulary and Immutable Identities

Invocation	One application-visible operation with one handle and terminal result.
Attempt	One bounded network decision generation inside an invocation.
ACK-closed snapshot	Immutable authenticated candidate set and its canonical digest.
Plan commit	One-shot generic projection of roles, dependencies, Providers, and opaque assignments.
Selection	ProviderToken-authorized final assignment sent to each chosen Provider.
Generation	Role execution epoch that fences late data and superseded work.

Invariant

Identity and state transitions, not timing assumptions, determine which work is current.

Next: start from the public application call

Public Application Call: Intent Only

```
handle = automatic_coordinator.request(  
    model=ModelRef(  
        model_name="Qwen/Qwen3-0.6B",  
        content_digest="sha256:<signed-model-manifest>",  
        semantics_digest="sha256:<tokenizer-and-preprocess>",  
    ),  
    task=InferenceTaskRef("text-generation"),  
    input=ApplicationInput.text("Explain named-data networking."),  
    options=GenerationOptions(max_new_tokens=64, temperature=0.0),  
    objective=OptimizationObjective.LOWEST_END_TO_END_LATENCY,  
    timeout_ms=120_000,  
)
```

- ▶ No Provider list, stage map, preloaded deployment, or precomputed split is required.
- ▶ The implemented model/task-first surface is `AutomaticPlanningCoordinator.request()`.

Next: separate human-readable model names from reuse authority

Model Identity: Name Is Discovery, Digests Are Authority

- ▶ **model name**: readable discovery, logging, and operator policy;
- ▶ **content digest**: canonical signed manifest binding weights, graph, and configuration;
- ▶ **semantics digest**: tokenizer, preprocessing, generation defaults, output interpretation;
- ▶ **source revision**: provenance only; never substitutes for either digest.

Invariant

Reuse requires exact content, semantics, graph, backend, precision, and runtime compatibility—not a matching model name.

Next: make model-specific knowledge replaceable

One Carrier, Multiple Model Families

Adapter port	Responsibility	Forbidden authority
Graph	canonical dependency snapshot	network or device mutation
Splitter	bounded graph-valid candidates	publication or Selection
Task	input/options/result schemas	placement or execution
State	lifecycle and exact state identity	cross-tenant access
Runner	execute accepted ONNX role	changing the plan or cross-Provider transport

The deployed runtime uses ONNX Runtime; PyTorch/Transformers are confined to a sealed offline exporter.

Invariant

An opaque model may expose one atomic node; it remains runnable but is never falsely split.

Next: derive placement structure from a real model graph

Target API: One Core, Two Delivery Lifecycles

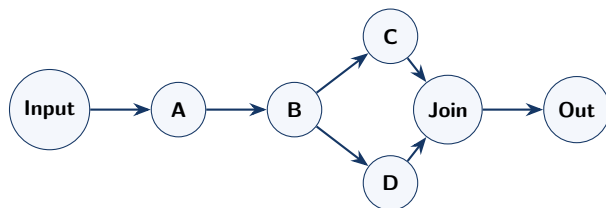
Dimension	Unary service (for example, one YOLO image)	Streamed generation (for example, an LLM)
Operation	one durable invocation	one durable invocation
Control plane	one ACK closure, plan commit, and Selection	the same; never repeated per token
Execution	one bounded forward pass	one prefill followed by bounded decode epochs
Visible delivery	no partial result; one terminal Response	ordered Event Data during execution, then one terminal Response
Provider state	short-lived inputs and outputs	persistent ORT session, exact local KV, sampler, and stream cursor
Public surface	<code>RequestService<Req,Resp></code>	<code>RequestServiceStreaming<Req,Event,Resp></code>

Invariant

Separate public entry points make lifetime and backpressure explicit; both remain one NDNSF invocation with the same discovery, authorization, placement, and terminal-result rules.

Next: derive placement structure from a real model graph

Dependency Graph: The Basis of a Legal Split



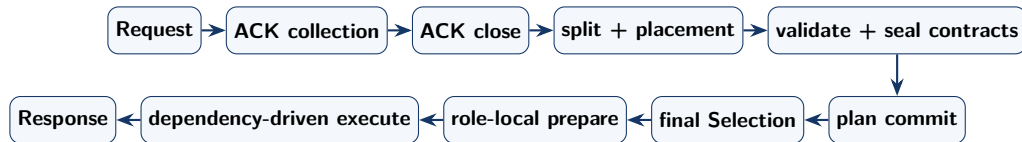
- ▶ nodes and topological order;
- ▶ tensor edges, dtype, shape, size estimate;
- ▶ legal cut edges and collective operations;
- ▶ model input/output contracts;
- ▶ canonical graph digest.

Invariant

Each tensor rank is a separate execution role assigned to exactly one Provider; the parallel group is metadata, not a cross-Provider role.

Next: follow one invocation end to end

Complete Lifecycle: One Request, One Final Plan



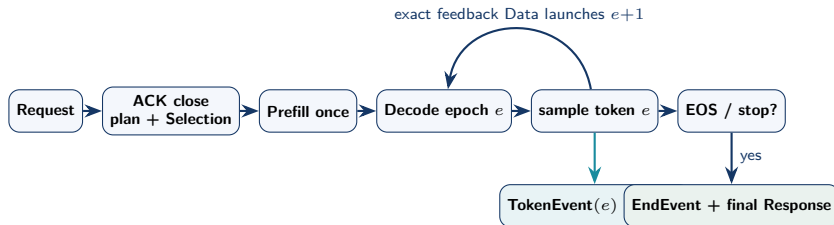
- ▶ Positive ACK means willingness and bounded capability, not role acceptance or model readiness.
- ▶ After Selection, each Provider fetches canonical ONNX components and assembles one local role.
- ▶ A role issues Interests for its declared inputs and runs after all are verified.

Invariant

No preparation-commit round and no global readiness barrier exist.

Next: inspect the request and offer phases

Target LLM Streaming: One Plan, Many Decode Epochs



- ▶ The last-stage role publishes internal feedback to Stage 0 and separately authorized Event Data to the User.
- ▶ Pre-expressed exact Interests and tokenEpoch provide ordered, duplicate-safe, resumable delivery; no control-plane round repeats per token.
- ▶ EOS, stop, bound, cancellation, or failure closes the stream; replanning creates a new attempt and stream epoch, and exactly one final Response wins.

Invariant

Autoregression is a bounded feedback loop inside one fixed plan, not one NDNFSF request or placement decision per token.

Request Envelope: Signed Intent and Bounded Work

- ▶ binds invocation ID, request ID, attempt, service, model identity, task, input manifest;
- ▶ binds deadline, security domain, encoding version, capability version, and acceptance predicate;
- ▶ payloads are canonical, bounded, schema-identified bytes;
- ▶ UserToken and normal NDNSF authorization protect the wire request.

Invariant

The request authorizes discovery for this exact operation; it does not authorize any Provider to execute.

Next: define what a positive ACK can safely claim

Signed Provider Offer: Capability Without Role Commitment

Resource and network view

- ▶ Provider identity and boot epoch;
- ▶ one execution slot/device, backend, usable memory;
- ▶ queue depth and wait estimate;
- ▶ measured RTT and bandwidth;
- ▶ expiry, capture time, units, confidence.

Reusable assets

- ▶ loaded role / assembled role / canonical component residency;
- ▶ lease/pin capability and generation;
- ▶ bounded reusable-state digests;
- ▶ artifact/runtime compatibility evidence.

Invariant

ACK evidence is fresh, signed, request-bound, and sanitized before policy code sees it.

Next: freeze the candidate set before planning

ACK Closure: The Planning Linearization Point

- ▶ ACK collection closes exactly once at the discovery deadline.
- ▶ The ordered successful candidate set receives a canonical digest.
- ▶ Late ACKs cannot mutate the snapshot.
- ▶ Repeated reads return the same snapshot.
- ▶ Tokens, keys, leases, and mutable Core handles are removed from the policy view.

Invariant

Every placement decision is reproducibly bound to one immutable ACK-closed digest.

Next: invoke one narrow, replaceable planning extension

Placement Extension: Pure Decision, No Side Effects

```
class ModelPlacementStrategy(ABC):  
    name: str  
    version: str  
    state_digest: str  
    @abstractmethod  
    def plan(self, request: PlacementRequest) -> PlacementDecision:  
        ...
```

- ▶ Input: model, graph, bounded candidates, sanitized Providers, network/catalog estimates, objective, and state contracts.
- ▶ Output: one certified split, one Provider per role, fallbacks, digests, and evidence.
- ▶ The implementation is local, allowlisted, digest-pinned operator-trusted code.

Invariant

Policy output is revalidated as untrusted data; the Python interface is not a sandbox.

Next: show the default policy

Default Policy: ACK-Driven PreSplitFirst

1. reject model, graph, backend, resource, freshness, and role mismatches;
2. prefer an exact feasible certified split candidate;
3. score loaded roles, assembled roles, canonical layers, network, and queue;
4. otherwise derive a new graph-valid split from the closed offers;
5. rank feasible bindings by predicted end-to-end critical-path cost;
6. break ties with canonical digests for deterministic replay.

Invariant

PreSplitFirst remains the default; layer reuse is one cost signal, not a separate LayerReuseFirst architecture.

Next: explain why byte size alone is unsafe

Capacity-Safe Splitting: Runtime Peak, Not File Size

$$M_{\text{peak}} = M_{\text{weights}} + M_{\text{activations}} + M_{\text{state}} + M_{\text{workspace}} + M_{\text{transient}} + M_{\text{margin}}$$

- ▶ each role must fit the Provider's usable envelope;
- ▶ graph cuts must preserve dependency and tensor contracts;
- ▶ homogeneous stages use deterministic near-equal balancing only when safe;
- ▶ missing safe bounds yields NO_FEASIBLE_SPLIT, not optimistic admission.

Invariant

“30 GB model / three 12 GB GPUs” is not a valid memory proof.

Next: turn cold-start cost into exact future reuse

Cache Residency: Evidence Has a Strength Order

LOADED ORT ROLE > ASSEMBLED ROLE > CANONICAL LAYERS >
REPOSITORY FETCH > COLD ASSEMBLY

- ▶ Residency is Provider state, not a property of the catalog.
- ▶ Boot epoch, cache generation, expiry, identity, precision, and runtime ABI must match.
- ▶ Loaded-runtime identity also binds Provider boot/process/device and assembler/ORT ABI.

Invariant

Warm reuse is an optimization guarded by exact evidence; clean loading remains the correctness fallback.

Next: separate immutable artifacts from volatile residency

Pre-Split Catalog and Distributed Repository

Catalog owns

- ▶ canonical ONNX graph/initializer/tokenizer manifests;
- ▶ certified split candidates without Provider bindings;
- ▶ role/rank interfaces, collective DAG, size and backend contracts;
- ▶ activation, retirement, and revocation snapshots.

Distributed repository owns

- ▶ signed named-object storage and retrieval;
- ▶ segmentation, checksums, replication evidence;
- ▶ deterministic canonical artifact Data names;
- ▶ Provider fetch before local ONNX assembly.

Invariant

Persistent model artifacts use the repository; request-scoped tensors use the producing Provider's namespace.

Next: seal the validated decision into the generic collaboration

Plan Commit: Immutable Role and Dataflow Projection

- ▶ binds the exact ACK-closed digest;
- ▶ every role appears once; every Provider receives at most one role;
- ▶ each assignment binds one RoleAssemblySpec;
- ▶ `mayPublish`, `mustFetch`, and `waitFor` form its RoleDataflowContract;
- ▶ the first valid commit digest is immutable;
- ▶ failed validation emits zero Selection messages.

Invariant

Plan commit is not Provider acceptance and does not consume a ProviderToken.

Next: make each final Selection crash-atomic

Final Selection: Complete Role Authority

- ▶ exactly one complete execution role per selected Provider;
- ▶ rank, assembly recipe, canonical components, dependencies, budgets, generation;
- ▶ exact publish/fetch/wait name templates and collective rounds;
- ▶ protected by the selected Provider's one-time ProviderToken;
- ▶ Provider verifies request, attempt, plan, role, artifact, dataflow, and security bindings;
- ▶ GPU execution-slot admission occurs JIT, not during ACK.

Invariant

Execution authority begins only after the exact Selection is durably accepted.

Next: prepare each role independently

Role-Local Preparation: No Global Barrier

Latch	Meaning
Selection accepted	exact role authority is durable
Local model ready	canonical components, RoleAssemblySpec, ORT ABI, JIT slot, and state verified
Direct inputs ready	every exact Interest target in waitFor verified

A role becomes runnable only when all three latches hold for the same attempt, plan, and generation.

Invariant

Each Provider assembles one local ONNX role; consumers pull every cross-Provider input by Interest.

Next: show the data-driven trigger

Data-Driven Execution: Consumer Pulls by Interest

$$\text{Runnable}(r, g) = \text{Accepted}(r, g) \wedge \text{LocalReady}(r, g) \wedge \bigwedge_{d \in \text{DirectInputs}(r)} \text{Verified}(d, g)$$

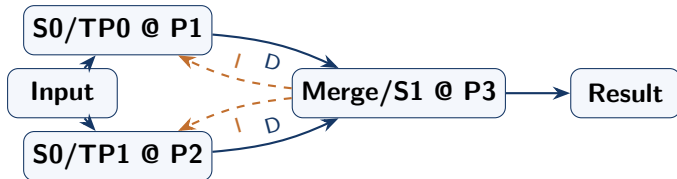
- ▶ `mayPublish`: exact output name templates owned by this role;
- ▶ `mustFetch`: exact Interests this role must express;
- ▶ `waitFor`: exact ALL/ANY/quorum readiness rule (collectives normally ALL);
- ▶ `retransmission` re-expresses the same name; stale attempt/plan/group data fails closed.

Invariant

No hidden TCP/RPC push, shared file, or cross-Provider opaque NCCL path may replace this contract.

Next: generalize beyond a linear pipeline

One Scheduler for Pipeline and Tensor Parallel



I = exact-name Interest issued by the consumer role; D = signed tensor Data returned by the producer role.

- ▶ each rank is a distinct role on one Provider;
- ▶ each round names producer, group/epoch, operation, rank, tensor, and segment;
- ▶ peers issue Interests for exact partials and advance only after the declared set arrives;
- ▶ missing rank, stale epoch, ambiguous producer, or unsatisfied join is rejected/fails closed.

Invariant

A parallel group is metadata; if merge needs computation, merge is another execution role.

Intermediate Tensors: Deterministic NDN Names

Producer-owned namespace

```
/<producer>/NDNSF/DI/TENSOR/<request>/<attempt>/<plan>/  
<group>/<epoch>/<op>/<round>/<src-role>/<tensor>/<microbatch>/<digest>/<segment>
```

- ▶ consumer first fetches a signed TensorObjectManifest;
- ▶ manifest binds producer/consumer contract, dtype/shape/layout, segment count, and digests;
- ▶ consumer pipelines exact segment Interests and verifies before reassembly;
- ▶ duplicate/out-of-order Data is safe; wrong name, signer, digest, or epoch fails closed.

Invariant

Request ID alone is insufficient: attempt, plan digest, group epoch, operation, rank, and segment prevent stale reuse.

Next: define exactly one admissible final result

Result Contract: Events Are Not the Final Answer

- ▶ sealed before execution: terminal role(s), aggregation rule, output schema, acceptance predicate;
- ▶ binds model/task/input/plan/attempt identities and result digest;
- ▶ unary execution publishes no partial result and completes with one Response;
- ▶ streaming execution may publish ordered, attempt-bound Event Data, but no event is the terminal result;
- ▶ EOS, stop, token bound, cancellation, or failure produces a terminal stream state; Response is accepted only after the complete result passes the sealed contract.

Invariant

Streaming changes when output becomes visible; it does not weaken the complete-result contract or the single terminal-Response invariant.

Next: separate durable intent from bounded retransmission

Invocation and Attempt: Why Retries Exist

- ▶ **Invocation:** the user's one durable call and public outcome.
- ▶ **Attempt:** one candidate snapshot, plan, Selection set, and generation namespace.
- ▶ Lost Request/ACK/Selection retries identically; tensor loss re-expresses the same Interest name.
- ▶ A different Provider assignment or split requires a new attempt.
- ▶ At most one terminal result wins for the invocation.

Invariant

A second attempt is recovery inside the same call; it is not a second application request.

Next: compose independent state machines

Orthogonal State Machines, One Auditable History

Machine	Principal states
Invocation	open → terminal success / failure / cancelled / expired
Attempt	discovering → ACK closed → planned → executing → terminal
Plan commit	open → committing → committed or terminal failure
Provider transaction	none → prepared → applied → finalized / rolled back
Role	selected → preparing → ready → running → terminal
Interest/Object	expected → expressed/fetching → verified or rejected
Parallel group	planned → round n → complete or epoch-failed

Invariant

Every transition is monotonic, digest-bound, deadline-bound, and replay-safe.

Next: classify failures by the authority that can react

Failure Taxonomy and Required Reaction

Failure class	Required reaction
Invalid offer, split, or policy output	exclude candidate; emit no Selection
Canonical fetch / local assembly failure	fail role/attempt; never emit partial downstream output
Selection delivery unknown	reconcile or retry identical protected bytes
Local load/state miss	use sealed clean-compute fallback or replan
Interest timeout/corruption/rank loss	fail affected role/group/attempt; fence the epoch
Provider crash/partition	fence generation; reconcile durable transaction
Result mismatch	reject Response; terminal failure or new attempt

Next: bound replanning and compensation

Replanning, Compensation, and Old-Output Adoption

- ▶ Replanning creates a new attempt with new assignment and generation identities.
- ▶ Old roles are cancelled or fenced; reservations and pins are released.
- ▶ Already published outputs are not silently reused.
- ▶ Adoption is allowed only if the new plan explicitly accepts the exact object contract and digest.
- ▶ Irreversible external side effects require application-level compensation.

Invariant

Recovery never lets a late old output cross an attempt boundary by accident.

Next: make cancellation and deadlines converge

Cancellation, Deadlines, and Resource Release

- ▶ one total invocation deadline continues through discovery, planning, fetch, assembly, loading, and execution;
- ▶ cancellation fences planner results, future commits, roles, objects, and Response;
- ▶ the first durable terminal state wins through compare-and-set;
- ▶ release covers execution-slot admission, cache pins, group keys, local assembly, tensor segments, and handles;
- ▶ unresolved delivery becomes explicit UNKNOWN, never implicit success.

Invariant

Safety is preserved even when liveness is lost under a permanent partition.

Next: trace the security chain

End-to-End Security Chain

1. Controller-signed encrypted permissions authorize requester and Provider identities.
2. NAC-ABE routes Request/Selection to service attributes and ACK/Response to permission attributes.
3. UserToken binds ACK and Response to the request.
4. one-time ProviderToken binds final Selection to the chosen Provider.
5. a Provider-specific grant binds request, attempt, plan core, policy snapshot, and protection epoch.
6. group capabilities disclose only the local wrapped epoch key; keys/nonces bind the exact role, peer, tensor manifest, and Data name.
7. ProtectedRuntime rejects mismatches before ORT and zeroizes device then host plaintext on cancel/revoke.

Invariant

No planning optimization bypasses NDNFSF authorization, token, signature, or replay controls.

Next: limit what resource and cache evidence reveals

Privacy of Resource and Cache Metadata

- ▶ expose bounded summaries, not device handles or unrestricted inventories;
- ▶ publish digests, tier, bytes, freshness, and capability—not prompts, token IDs, plaintext state, or tenant names;
- ▶ retain raw tokens and keys only in protected runtime state;
- ▶ encrypt protected Selection retry material at rest and expire it;
- ▶ deny cross-tenant inference-state reuse by default.

Invariant

The placement policy receives enough evidence to decide, but not enough authority to inspect or mutate Provider state.

Next: treat KV as one exact state profile

KV Cache Support Without an LLM-Bound API

- ▶ KV is one EXACT_PREFIX_REUSABLE state profile.
- ▶ Identity binds model/semantics, adapter/runner, split/layers, exact prefix tokens, positions, layout, precision, ABI, security domain, and epochs.
- ▶ Prefill consumes the full prompt once; each decode epoch consumes one new token and the exact Provider-local KV state.
- ▶ Missing or incompatible coverage falls back to clean recomputation in a new fenced attempt.
- ▶ Cross-Provider migration is disabled unless an adapter supplies an explicit encrypted migration contract.

Invariant

State reuse may reduce work; it may never change the accepted result.

Next: separate perceived latency from decode throughput

Continuous Generation: Optimize TTFT and TPOT

Critical-path metrics

TTFT = discovery + planning + role preparation + prefill + first Event Data

TPOT $\approx$ stage ORT + activation Interest/Data + sampling + feedback Interest/Data

20 token/s target $\Rightarrow$ TPOT $\leq$ 50 ms

Optimization order

1. keep ORT sessions, device buffers, and I/O bindings resident;
2. prefill once; decode one token with exact local KV;
3. pre-express exact activation, feedback, and Event Interests;
4. sign/publish through a bounded asynchronous queue with backpressure;
5. grant one stream content key, then protect events symmetrically—no per-token ABE bootstrap.

Invariant

Streaming improves perceived latency; only incremental KV decode and a shorter per-epoch path improve tokens/s. This is a target, not current performance evidence.

Observability: Evidence for Decisions and Execution

- ▶ planning input digest, strategy identity, candidate rejection reasons, selected cost components;
- ▶ ACK closure, plan commit, Selection transaction, role readiness, object verification, and terminal status;
- ▶ canonical fetch, local assembly, ORT load, Interest wait/retransmit, collective rounds, compute, TTFT, latency, tokens/s;
- ▶ sampled request-correlated traces with no plaintext prompt or secret material;
- ▶ `report_operation_status()` reports authenticated progress, not execution authority.

Next: show the implemented generic carrier calls

Implemented Carrier API

```
invocation = service_user.begin_collaboration(  
    service_name, opaque_request,  
    mode="DEFERRED",  
    ack_timeout_ms=ack_ms, timeout_ms=total_ms,  
)  
closed = await invocation.acks_closed()  
decision = coordinator.plan(closed)  
sealed = await coordinator.validate_materialize_and_seal(decision)  
await invocation.commit_plan(  
    ack_closed_digest=closed.digest,  
    roles=sealed.roles,  
    dependencies=sealed.dependencies,  
    provider_assignments=sealed.providers_by_role,  
    assignment_payloads=sealed.role_assembly_and_dataflow_contracts,  
)
```

Invariant

The compatibility method name does not imply User-side role materialization:
Providers assemble ONNX roles from sealed contracts.

Next: show operator configuration

Operator Configuration: Policy, Not Deployment

```
schema: ndnsf-di-application/v2
identity:
  requester: /ndnsf/di/user
  controller: /ndnsf/controller
repository:
  prefix: /ndnsf/di/models
planning:
  strategy: {name: pre-split-first, version: "3",
    digest: "sha256:<strategy-code-and-state>"}
  gpu_safety_margin: 0.15
cache:
  canonical_onnx_components: {retain_verified: true}
  derived_state:
    exact_prefix_local_reuse: true
    cross_tenant_reuse: false
    migration: disabled
```

No model, Provider list, role graph, private key, or precomputed deployment belongs here.

Next: map the design to current code

Target Surface and Current Migration Boundary

Area	Current mechanism or explicit target boundary
Generic carrier	deferred begin, immutable ACK closure, one-shot plan commit
Planning	common graph/candidate values, read-only catalog snapshot, external allowlist, default pre-split-first policy
Adapters	composed graph/split/task/state/runner ports with digest pins
Artifacts	canonical ONNX graph/initializer manifests; repository store/fetch; Provider-local assembly
Execution	one role per Provider; Interest pull; manifest/segment and collective-round gates
State	exact prefix-KV identity, local reuse binding, clean-compute/replan fallback
Delivery	unary terminal Response exists; request-scoped ordered Event Data plus one terminal Response is the target streaming extension
Recovery/security	durable selection transaction, generations, cancellation, tokens, NAC-ABE

Invariant

Multi-role Provider, cross-Provider logical role, DEVICE_SET, and LayerReuseFirst-default paths are migration drift, not target completion evidence.

Validation Pyramid: Cheapest Correct Gate First

1. pure contract and negative unit tests;
2. native build and binding checks;
3. one-container security/control smoke with tiny payload;
4. exact-SIF CPU small-ONNX pipeline/tensor/hybrid Interest/Data flows;
5. MiniNDN multi-Provider lifecycle, negative names/epochs, then Qwen complete answers;
6. optional Slurm/Apptainer network, CUDA, stage, and campaign gates.

Invariant

A higher-cost test does not repair a failed lower gate, and a lower gate does not prove a higher-layer claim.

Next: report the accepted local evidence

Current Evidence and the Remaining Protocol Gap

Already exercised

- ▶ deferred collaboration and immutable ACK closure;
- ▶ one Provider / one role commit and Selection checks;
- ▶ exact tensor names, signed manifest/segments, and same-name retry;
- ▶ verified-input runtime gates and protected grant/capability/zeroization negatives;

Must still be closed

- ▶ one full four-Provider/four-role lifecycle;
- ▶ deterministic drop/reorder/duplicate/replay/cancel packet matrix;
- ▶ exact-SIF CPU small-ONNX and MiniNDN gates;
- ▶ request-scoped event ordering, backpressure, resume, cancellation, and final-Response integration;
- ▶ CUDA/Qwen correctness and performance evidence.

Invariant

T011–T014 are locally green; network, full-lifecycle, and physical evidence remain separate gates.

Remaining Physical Requalification Boundary

- ▶ exact locally qualified SIF digest;
- ▶ actual allocated GPU model, UUID, memory, CUDA provider, and `cpuFallback=0`;
- ▶ allocation-scoped cross-node NFD connectivity;
- ▶ role assembly hashes, complete graph/initializer coverage, and tensor-edge receipts;
- ▶ exact standalone reference tokens and full distributed answers;
- ▶ warmup-separated TTFT, per-token latency, total latency, and tokens/s distributions.

Invariant

These are test-environment facts, not missing semantics in the NDNFSF-DI lifecycle.

Next: build the local container safely

Local SIF Step 1: Build Inside the Target ABI

- ▶ The release candidate begins as one complete locally built Apptainer SIF.
- ▶ Build native Python/C++ extensions inside that SIF or an ABI-identical sealed builder.
- ▶ Record source seal, base digest, Apptainer, Python SOABI, glibc, Boost, ndn-cxx, NDN-SVS, ORT/CUDA, and SIF SHA-256.
- ▶ Never copy a host-built `_ndnsf.so` into the candidate.
- ▶ Keep models, NDN keys, credentials, and evidence outside the SIF.

Invariant

The deployment SIF contains ONNX Runtime, not PyTorch/Transformers; those belong only in the offline exporter.

Next: qualify the exact local candidate

Local SIF Step 2: Qualify the Exact Candidate

- ▶ Import `ndnsf._ndnsf`; run the real MiniNDN runner `-help`.
- ▶ Verify extension/native hashes, RPATH/ldd closure, SOABI, library versions, ORT providers, and absolute artifact paths.
- ▶ Run CPU small-ONNX Request/ACK/Selection and Provider-local assembly.
- ▶ Run pipeline and two-rank tensor Interest/Data flows plus negative name/epoch/rank tests.
- ▶ Reject stale extensions, host-library leakage, unresolved symbols, and cwd-dependent artifacts.

Invariant

Host tests or a different SIF cannot qualify the file that will run on Tiger.

Next: discover live cluster facts before upload

iTiger Step 1: Read-Only Preflight

```
uofm-vpn-status  
ssh -o BatchMode=yes itiger 'hostname; whoami'  
  
tools/ndnsf-di/ndnsf-di-itiger-qwen discover --help
```

- ▶ Recheck quota, partition, GRES spelling, Apptainer, driver, addresses, and scratch.
- ▶ Keep durable releases/models/evidence under /project/\$USER/ndnsf-di.
- ▶ Never build, convert, download a large model, or infer on the login node.

Next: upload the already-qualified immutable SIF

iTiger Step 2: Hash-Promote the Local SIF

1. Upload the locally qualified SIF to project staging; do not rebuild/materialize another candidate on Tiger.
2. Verify source seal, manifest, and SIF SHA-256; atomically promote the exact file.
3. Preflight sbatch: absolute SIF/bundle/model/evidence paths, cwd, binds, partition/GRES, Provider identities, and output directory.
4. In a GPU allocation, run `apptainer exec -nv`; verify device, driver, ONNX Runtime CUDA, NFD, and NDNSF imports.
5. Assert PyTorch/Transformers are absent and CPU fallback is disabled.

Invariant

Host PASS, exact local-SIF PASS, and Tiger GPU PASS are separate gates bound to one SIF hash.

Next: advance through the Qwen test ladder

iTiger Step 3: Network, Reference, Stages, Then DI

1. **multi-node NFD probe**: no model; prove job-scoped faces and signed Data exchange;
2. **standalone reference**: canonical ONNX, tokenizer, prompt set, greedy outputs;
3. **role load**: one Provider per role, exact graph/initializer coverage, ORT CUDA, no CPU fallback;
4. **full development invocation**: request/attempt/plan/group, assembly, Interest/Data edge and collective receipts, complete answer;
5. **formal campaign**: frozen SIF/model/strategy/Provider count; warmup-separated repetitions.

Invariant

Run stages in this order; stop at the first failed gate and preserve it.

Next: retain evidence and enforce stop rules

iTiger Step 4: Evidence, Distribution, and Stop Rules

- ▶ bind model/tokenizer, source, exact SIF digest, plan/group, role map, GPU UUIDs, job/run/repetition IDs;
- ▶ retain full answers, tokens, EOS, TTFT, latency, tokens/s, assembly, Interest wait/retransmit, collective and transfer timing;
- ▶ retain failed, timed-out, cancelled, mismatched, and truncated repetitions; exclude them from success summaries without deleting them;
- ▶ never silently retry a measured identity, change GPU count/type, quantize, extend time, or overwrite a failure;
- ▶ promote evidence before allocation scratch is removed.

Invariant

One warm run is an anecdote; the frozen repeated campaign is the distribution.

Next: close with the governing design principle

Persistent Artifacts vs. Request-Scoped Tensors

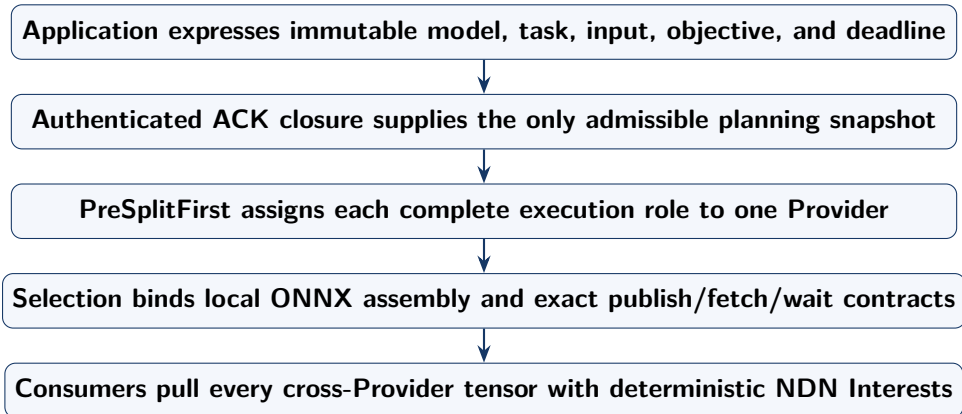
- ▶ DistributedRepo owns persistent canonical ONNX graph/initializer/tokenizer objects: queued store, manifest trust, segmented Data, receipts, activation, recovery, and GC.
- ▶ Request-scoped activations and collective partials are published under the producing Provider's namespace and pulled directly by exact Interests.
- ▶ Both paths use signed segmented NDN Data, but their names, deadlines, authorization, freshness, and retention differ.
- ▶ Repo success does not prove pipeline/tensor dataflow; the corrected protocol needs its own exact-SIF and MiniNDN gates.

Required separation

Model artifacts are durable and content-addressed. Intermediate tensors are attempt/plan/group-epoch bound, short-lived, and accepted only by roles whose sealed `mustFetch/waitFor` contracts name them.

Next: close the corrected protocol before large-model performance claims

Final Takeaway



One Provider/role; ONNX local compute; NDN pull across Providers.